

MINI-PROJECT REPORT

Protocol Analysis with Wireshark

Unit 2: Network Models and Protocols

Student Name	Shubham Shinde
PRN	2125UCSM1042
Course / Module	Unit 2: Network Models and Protocols
Project Title	Protocol Analysis with Wireshark (NPC Assignment 2)
Tools / Environment	Wireshark (v4.x) on Kali Linux (eth0)
GitHub Repository	Protocol-Analysis-with-Wireshark
GitHub Link	https://github.com/ShubhamShinde148/Protocol-Analysis-with-Wireshark

1. Aim

To capture and analyze network packets using Wireshark, identify the TCP three-way handshake (SYN, SYN-ACK, ACK), and observe the subsequent HTTP GET request.

2. Software and Requirements

- Kali Linux
- Wireshark (open-source network protocol analyzer)
- Mozilla Firefox or another web browser
- Internet connection
- Active network interface: eth0

3. Procedure

1. Open Wireshark in Kali Linux and select the active network interface (eth0).
2. Start packet capture.
3. Open a web browser and navigate to <http://neverssl.com>.
4. Wait for the webpage to load, then stop the packet capture.
5. Apply the display filter 'http' to locate HTTP traffic.
6. Identify the HTTP GET request and its corresponding TCP connection.
7. Analyze the TCP three-way handshake: SYN, SYN-ACK, and ACK.
8. Record the source and destination IP addresses and TCP port numbers.
9. Save the packet capture in .pcapng format.

4. TCP Three-Way Handshake Observation

The captured TCP connection shows the standard three-way handshake used to establish a reliable TCP connection before HTTP data is exchanged.

Packet	Packet No.	Source IP	Source Port	Destination IP	Destination Port
SYN	87	192.168.17.130	34078	34.223.124.45	80
SYN-ACK	103	34.223.124.45	80	192.168.17.130	34078
ACK	104	192.168.17.130	34078	34.223.124.45	80

5. HTTP GET Request

After the TCP connection was established, the client sent an HTTP GET request to the server. The observed request was:

GET / HTTP/1.1

The HTTP server responded successfully with an HTTP 200 OK response.

Packet	Source IP	Source Port	Destination IP	Destination Port
HTTP GET (105)	192.168.17.130	34078	34.223.124.45	80

6. Transport Layer Analysis

The transport-layer protocol used in this communication is TCP. The client used the ephemeral source port 34078, while the web server listened on destination port 80, the standard port for HTTP. The TCP handshake established the connection before the HTTP GET request was transmitted.

7. Result

The network traffic was successfully captured and analyzed using Wireshark. The TCP three-way handshake and the subsequent HTTP GET request were identified successfully. The source and destination IP addresses and TCP port numbers were recorded from the capture.

8. Conclusion

This practical demonstrated how Wireshark can be used to inspect network protocols at the packet level. The analysis showed that a TCP connection is established using SYN, SYN-ACK, and ACK packets, after which the client can send an HTTP GET request over TCP port 80. Wireshark provides useful visibility into IP addresses, transport-layer ports, TCP flags, and application-layer HTTP traffic.

9. Screenshots

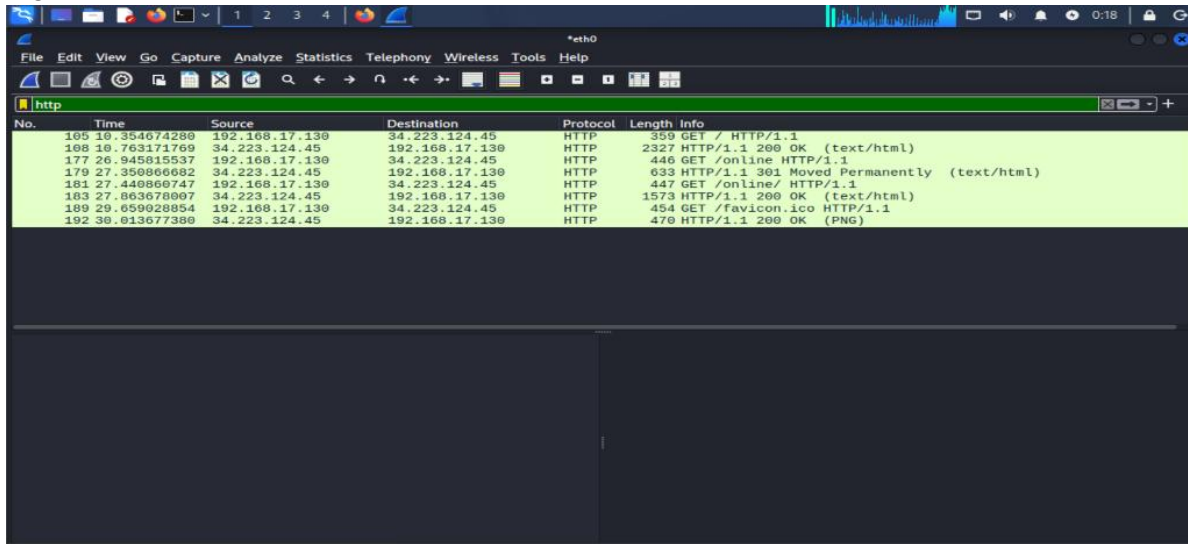
The following screenshots should be inserted in this section in the final submission:

Figure 1: TCP three-way handshake and HTTP GET packet (Packet 87, 103, 104, and 105).

No.	Time	Source	Destination	Protocol	Length	Info
105	10.354674288	192.168.17.130	34.223.124.45	HTTP	359	GET / HTTP/1.1
108	10.763171769	34.223.124.45	192.168.17.130	HTTP	237	HTTP/1.1 200 OK (text/html)
87	8.988100465	192.168.17.130	34.223.124.45	TCP	74	34078 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TS...
104	9.959302180	192.168.17.130	34.223.124.45	TCP	74	80 → 34078 [ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
103	10.352589629	34.223.124.45	192.168.17.130	TCP	60	80 → 34078 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
104	10.352895221	192.168.17.130	34.223.124.45	TCP	54	34078 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
106	10.356134346	34.223.124.45	192.168.17.130	TCP	60	80 → 34078 [ACK] Seq=1 Ack=306 Win=64240 Len=0
109	10.763999352	192.168.17.130	34.223.124.45	TCP	54	34078 → 80 [ACK] Seq=306 Ack=2274 Win=65535 Len=0
153	15.084626190	34.223.124.45	192.168.17.130	TCP	60	80 → 34078 [FIN, PSH, ACK] Seq=2274 Ack=306 Win=64240 Len=0
154	15.896787710	192.168.17.130	34.223.124.45	TCP	54	34078 → 80 [FIN, ACK] Seq=306 Ack=2275 Win=65535 Len=0
155	15.897395596	34.223.124.45	192.168.17.130	TCP	60	80 → 34078 [ACK] Seq=2275 Ack=307 Win=64239 Len=0

Frame 105: Packet, 359 bytes on wire (2872 bits), 359 bytes captured	0000	00 50 50 f6 da 31 00 0c 29 2b 76 ae 08 00 45 00	EPV...1.. }+*
Ethernet II, Src: VMware_2b:76:ae (00:0c:29:2b:76:ae), Dst: VMware_1	0010	01 59 a7 57 40 00 40 06 21 11 c0 a8 11 82 22 df	Yw@ @. f... Pv2 ...
Destination: VMware_f6:da:31 (00:50:56:f6:da:31)	0020	7c 2d 85 1c 00 50 76 5a ac f0 3e 50 00 98 50 18	[-... GE T...
.....0..... = LG bit: Globally unique address	0030	fa f0 72 82 00 00 47 45 54 20 2f 20 48 54 54 50	r...GE T...
.....0..... = LG bit: Individual address (uni	0040	2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 6c 75 73 68	/1.1..Ho st
Source: VMware_2b:76:ae (00:0c:29:2b:76:ae)	0050	02 72 69 67 68 74 73 69 6c 76 65 72 6d 65 6c 6f	brightsi lv
.....0..... = LG bit: Globally unique address	0060	64 79 2e 6e 65 76 65 72 73 73 6c 2e 63 6f 6d 0d	dy.never ss
.....0..... = LG bit: Individual address (uni	0070	0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d 6f 7a	User-Ag en
Type: IPv4 (0x0800)	0080	69 6c 6c 61 2f 35 2e 30 20 20 58 31 31 3b 20 4c	illa/5.0 (-
[Stream index: 0]	0090	69 6e 75 78 20 78 38 36 5f 36 34 3b 20 72 76 3a	linux x86 _6
Internet Protocol Version 4, Src: 192.168.17.130, Dst: 34.223.124.45	00a0	31 34 30 2e 30 29 20 47 65 63 6b 6f 2f 32 30 31	140.0) G ec
Transmission Control Protocol, Src Port: 34078, Dst Port: 80, Seq: 1	00b0	30 30 31 30 31 20 40 69 72 65 60 6f 70 2f 31 34	00101 F1 re
Hypertext Transfer Protocol	00c0	30 2e 30 0d 0a 41 63 63 65 70 74 3a 20 2a 2f 2a	0.0. Acc ep
	00d0	0d 0a 41 63 63 65 70 74 2d 4c 61 6e 67 75 61 67	.Accept -L
	00e0	65 3a 20 65 6e 2d 55 53 2c 65 6e 3b 71 3d 30 2e	e: en-US ,e
	00f0	35 0d 0a 41 63 63 65 70 74 2d 45 6e 63 6f 64 69	S' Accep t-i
	0100	6c 67 3a 20 67 7a 69 70 2c 20 64 65 66 6c 61 74	ng: gzip ,

Figure 2: HTTP GET request and HTTP 200 OK response.



No.	Time	Source	Destination	Protocol	Length	Info
105	10.354674280	192.168.17.130	34.223.124.45	HTTP	399	GET / HTTP/1.1
108	10.763171769	34.223.124.45	192.168.17.130	HTTP	2327	HTTP/1.1 200 OK (text/html)
177	26.945815537	192.168.17.130	34.223.124.45	HTTP	446	GET /online HTTP/1.1
179	27.350866682	34.223.124.45	192.168.17.130	HTTP	633	HTTP/1.1 301 Moved Permanently (text/html)
181	27.440869747	192.168.17.130	34.223.124.45	HTTP	447	GET /online/ HTTP/1.1
183	27.863678907	34.223.124.45	192.168.17.130	HTTP	1573	HTTP/1.1 200 OK (text/html)
189	29.659028854	192.168.17.130	34.223.124.45	HTTP	454	GET /favicon.ico HTTP/1.1
192	30.013677380	34.223.124.45	192.168.17.130	HTTP	470	HTTP/1.1 200 OK (PNG)

10. Submission Evidence Checklist

- ✓ GitHub Repository: <https://github.com/ShubhamShinde148/Protocol-Analysis-with-Wireshark>
- ✓ Wireshark Packet Capture file: protocol_analysis.pcapng
- ✓ TCP Handshake Screenshot: Figure_1_TCP_Handshake.png
- ✓ HTTP Traffic Screenshot: Figure_2_HTTP_Traffic.png
- ✓ Project Documentation Report: Protocol_Analysis_with_Wireshark_Report.docx & PDF